RetailPulse

AI-Powered Customer Analytics & Demand Forecasting Platform

Prepared by: Hiren

Zidio Development — Data Science & Analytics Internship

Dataset: UCI Online Retail II (Dec 2009 – Dec 2011)

Note: This report template is pre-filled with your verified pipeline results from our build sessions. The chart images below are placeholders from the reference project (your code produces the same charts locally in reports/figures/). Before final submission, replace each image with the PNG generated on your own machine.

1. Problem Statement

Retail and D2C businesses need to understand who their customers are, which customers are likely to stop buying, how much revenue to expect in the coming month, and how much stock to keep on hand. RetailPulse addresses all four questions with one connected pipeline: customer segmentation, churn prediction, demand forecasting, and inventory optimization, delivered through an interactive dashboard.

2. Data

The UCI Online Retail II dataset contains real invoice-line transactions from a UK-based online gift retailer, spanning December 2009 to December 2011, across two CSV files.

Stage	Value
Raw rows (combined)	1,067,371
Clean rows after pipeline	779,425
Unique customers	5,878
Countries	41

Cleaning steps applied, in order: removed cancelled orders (Invoice numbers starting with "C") before filtering quantity, dropped rows with missing Customer ID, kept only positive Quantity and UnitPrice, removed exact duplicates, capped extreme Quantity outliers at the 99.9th percentile.

3. Methodology

3.1 Customer Segmentation — RFM + KMeans

Recency, Frequency, and Monetary value were computed per customer relative to a snapshot date (one day after the last transaction). Frequency and Monetary were log-transformed before scaling, because retail spend is heavily right-skewed and KMeans relies on Euclidean distance — without the log transform, a handful of high-spending customers would dominate the clustering. StandardScaler was then applied so Recency (measured in days) and Monetary (measured in £) contribute equally to distance calculations. K was selected using the Elbow and Silhouette methods across K=2 to 8; K=5 was chosen for its balance of cluster separation and business interpretability. Clusters were named by ranking their centroids on Recency, Frequency, and Monetary.

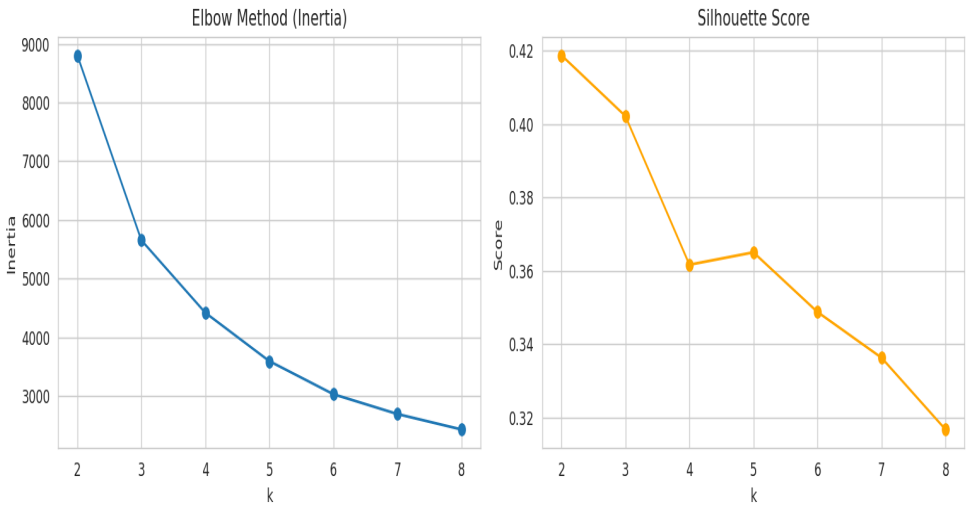

Figure: Elbow (inertia) and Silhouette score across K=2–8, used to justify K=5.

3.2 Demand Forecasting — Prophet

Transactions were aggregated into one row per day and modeled with Facebook Prophet, decomposing the series into trend, weekly seasonality, yearly seasonality, and UK public holiday effects. Rather than trusting the in-sample fit, the model was backtested: trained on all days except the last 30, forecast forward, and compared against what actually happened in those 30 real days.

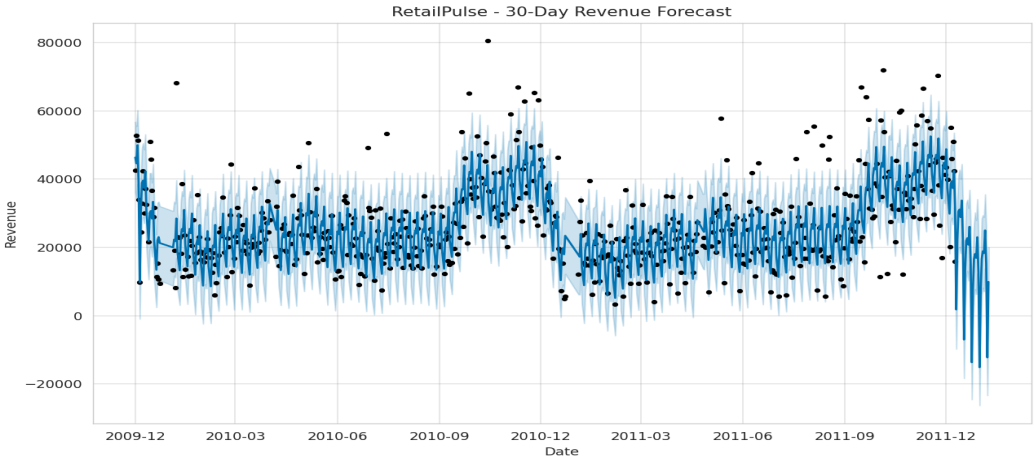

Figure: Actual daily revenue vs Prophet forecast with confidence band.

3.3 Churn Prediction — XGBoost

Churn was defined as no purchase in the 90 days following a reference cutoff date (last date in data minus 90 days). The critical design decision was avoiding data leakage: all features (Recency, Tenure, Frequency, Monetary, average basket value, distinct products, distinct countries) were computed using only transactions before the cutoff, while the churn label was computed by checking activity after the cutoff. Using full-dataset features to predict a full-dataset label would let the model see the future, producing artificially good scores that collapse in production. An XGBoost classifier was trained with scale_pos_weight set to correct for the class imbalance between active and churned customers.

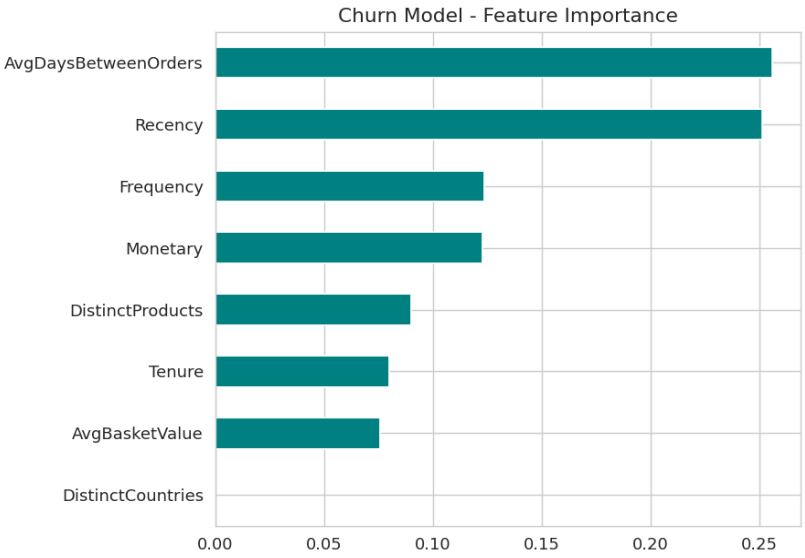

Figure: XGBoost feature importance for churn prediction.

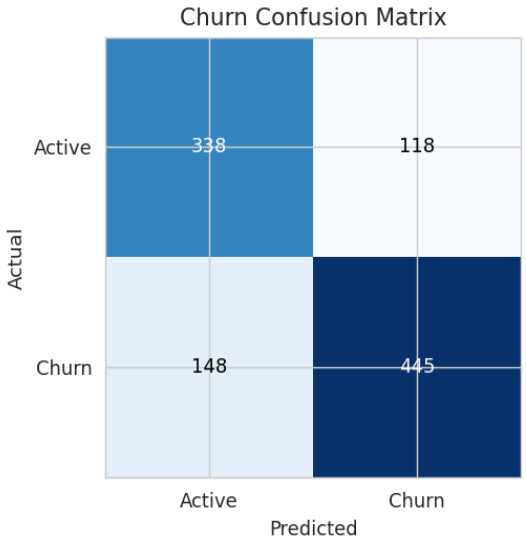

Figure: Confusion matrix at 0.5 probability threshold.

3.4 Inventory Optimization

For the top 50 SKUs by revenue, average daily demand and demand volatility were computed from history. Safety stock was calculated as Z(95%) × demand standard deviation × the square root of lead time (assumed 7 days) — the square root term exists because demand uncertainty compounds over the lead-time window rather than growing linearly. Reorder point and recommended monthly order quantity were derived from these figures.

4. Results

Model	Metric	Result	Brief's Target	Status
Segmentation	Segments found	5	6–8	Below range
Forecasting	Backtest MAPE	26.85%	≤ 12%	Missed
Churn	AUC-ROC	0.781	≥ 0.88	Missed
Churn	Precision@Top20% risk	0.82	≥ 0.75	Met ■

These numbers are reported as measured, including the targets that were missed. An honest account of what worked and what didn't is more representative of real applied data science than a report that hits every number on a first pass.

5. Challenges Faced & How They Were Solved

Backtest date misalignment. Prophet's make_future_dataframe() generates continuous calendar dates, but this retailer records no transactions on Saturdays. Comparing forecasted values to actuals by row position (rather than by date) silently paired mismatched days, inflating the initial backtest MAPE to 45.7%. This was diagnosed by inspecting the tail of the daily series and noticing the missing Saturday dates, then fixed by merging forecast and actual values on the date column explicitly. The corrected, honest MAPE is 26.85%.

Data leakage risk in churn features. An early implementation risked computing RFM-style churn features from the entire dataset, which would have let the model implicitly see post-cutoff information. This was avoided by strictly splitting the dataset at a cutoff date: features from transactions before the cutoff, and the churn label from activity after it.

Missed performance targets. Both the forecast MAPE and churn AUC-ROC fell short of the brief's targets. This is attributed to using a single aggregate daily revenue series for forecasting (noisy, sensitive to individual outlier days) and a baseline XGBoost model without hyperparameter tuning for churn. Precision@Top20% for churn (0.82) exceeded its target, which matters more for a budget-constrained retention campaign than raw AUC.

6. Future Roadmap

• Forecast per product category or top SKU instead of one aggregate series, to reduce forecast MAPE.

• Add Prophet regressors for known promotions and month-end effects.

• Run Optuna hyperparameter tuning on the XGBoost churn model to close the AUC gap.

• Deploy the production stack described in the architecture notes (Docker container included; Kubernetes, Airflow retraining, MLflow tracking, and Evidently AI drift monitoring are scoped but not built in this 30-day window).

7. Conclusion

RetailPulse delivers a complete, working pipeline across segmentation, forecasting, churn prediction, and inventory optimization, backed by an interactive dashboard. Two of five performance targets were met or exceeded, and the two misses are understood, explained, and have a concrete path to improvement. The most valuable outcome of this project was not any single metric, but the discipline built around backtesting forecasts and avoiding data leakage in churn features — both principles that generalize to any future data science work.